Computation Graphs, Backpropagation and Autodiff

How gradients flow through neural networks

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

Backprop vocabulary

The goal: two passes over one graph

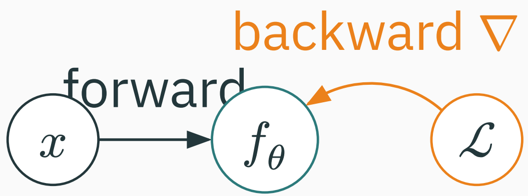
V VISUAL

Training is a **forward** pass and a **backward** pass over the same computation graph:

The goal: two passes over one graph

V VISUAL

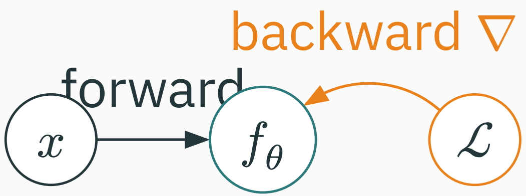
Training is a **forward** pass and a **backward** pass over the same computation graph:



The goal: two passes over one graph

V VISUAL

Training is a **forward** pass and a **backward** pass over the same computation graph:



forward: $x \to \mathcal{L}$ · **backward:** $\mathcal{L} \to \nabla_\theta \mathcal{L}$

Training needs gradients

Every update step is

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).$$

Training needs gradients

Every update step is

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).$$

A modern network has $P = 10^7$ to 10^{11} parameters.

Training needs gradients

Every update step is

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t).$$

A modern network has $P = 10^7$ to 10^{11} parameters.

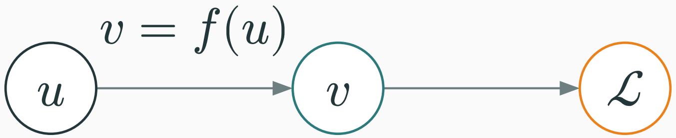
We need **all** P partial derivatives $\partial \mathcal{L} / \partial \theta_j$ — cheaply, and exactly. Backprop delivers them in **one backward pass**.

One local node

Zoom in on a single edge of the graph: $u \to v \to \mathcal{L}$.

One local node

Zoom in on a single edge of the graph: $u \to v \to \mathcal{L}$.



One local node

Zoom in on a single edge of the graph: $u \to v \to \mathcal{L}$.

$$u \xrightarrow{v = f(u)} v \longrightarrow \mathcal{L}$$

We want $\overline{u} := \partial\mathcal{L}/\partial u$, and we are **given** $\overline{v} := \partial\mathcal{L}/\partial v$ from the node ahead.

Upstream, local, downstream

D DERIVATION

The chain rule splits the work into two pieces we already have:

$$\underbrace{\partial\frac{\mathcal{L}}{\partial}u}_{\text{downstream}} = \underbrace{\partial\frac{\mathcal{L}}{\partial}v}_{\text{upstream}} \cdot \underbrace{\partial\frac{v}{\partial}u}_{\text{local}}$$

Upstream, local, downstream

D DERIVATION

The chain rule splits the work into two pieces we already have:

$$\underbrace{\partial\frac{\mathcal{L}}{\partial}u}_{\text{downstream}} = \underbrace{\partial\frac{\mathcal{L}}{\partial}v}_{\text{upstream}} \cdot \underbrace{\partial\frac{v}{\partial}u}_{\text{local}}$$

$$\boxed{\textbf{downstream} = \textbf{upstream} \times \textbf{local}}$$

Upstream, local, downstream

D DERIVATION

The chain rule splits the work into two pieces we already have:

$$\underbrace{\partial\frac{\mathcal{L}}{\partial}u}_{\text{downstream}} = \underbrace{\partial\frac{\mathcal{L}}{\partial}v}_{\text{upstream}} \cdot \underbrace{\partial\frac{v}{\partial}u}_{\text{local}}$$

$$\boxed{\textbf{downstream} = \textbf{upstream} \times \textbf{local}}$$

Upstream comes from the node ahead. **Local** is the derivative of **this** node alone. Their product flows **downstream**.

The rule, in bar notation

D DERIVATION

Write $\overline{u} = \partial \mathcal{L} / \partial u$. Then every node obeys

$$\overline{u} = \overline{v} \cdot \frac{\partial v}{\partial u}.$$

The rule, in bar notation

DERIVATION

Write $\overline{u} = \partial \mathcal{L} / \partial u$. Then every node obeys

$$\overline{u} = \overline{v} \cdot \frac{\partial v}{\partial u}.$$

The node only needs to know its **own** local derivative $\partial v / \partial u$. It multiplies whatever gradient $\overline{v}$ arrives from above — no global bookkeeping.

Example: a square node

D DERIVATION

Node $v = u^2$, so the local derivative is $\partial v/\partial u = 2u$.

Example: a square node

D DERIVATION

Node $v = u^2$, so the local derivative is $\partial v / \partial u = 2u$. Suppose $u = 3$ (forward) and the upstream gradient is $\overline{v} = 7$. Then

$$\overline{u} = \overline{v} \cdot 2u = 7 \cdot 6 = 42.$$

Example: a square node

D DERIVATION

Node $v = u^2$, so the local derivative is $\partial v/\partial u = 2u$. Suppose $u = 3$ (forward) and the upstream gradient is $\overline{v} = 7$. Then

$$\overline{u} = \overline{v} \cdot 2u = 7 \cdot 6 = 42.$$

downstream $\overline{u} = 42$

Example: a square node

D DERIVATION

Node $v = u^2$, so the local derivative is $\partial v / \partial u = 2u$. Suppose $u = 3$ (forward) and the upstream gradient is $\overline{v} = 7$. Then

$$\overline{u} = \overline{v} \cdot 2u = 7 \cdot 6 = 42.$$

downstream $\overline{u} = 42$

The node did **one** thing: multiply the incoming $\overline{v} = 7$ by its own local slope $2u = 6$.

Addition node: copy the gradient

D DERIVATION

Node $v = a + b$. Local derivatives $\partial v / \partial a = 1$, $\partial v / \partial b = 1$.

Addition node: copy the gradient

D DERIVATION

Node $v = a + b$. Local derivatives $\partial v / \partial a = 1$, $\partial v / \partial b = 1$.

$$\overline{a} = \overline{v} \cdot 1 = \overline{v}, \qquad \overline{b} = \overline{v} \cdot 1 = \overline{v}.$$

Addition node: copy the gradient

D DERIVATION

Node $v = a + b$. Local derivatives $\partial v/\partial a = 1$, $\partial v/\partial b = 1$.

$$\overline{a} = \overline{v} \cdot 1 = \overline{v}, \qquad \overline{b} = \overline{v} \cdot 1 = \overline{v}.$$

$+$ **copies the upstream gradient to every input**

Multiplication node: swap the inputs

D DERIVATION

Node $v = ab$. Local derivatives $\partial v/\partial a = b$, $\partial v/\partial b = a$.

Multiplication node: swap the inputs

D DERIVATION

Node $v = ab$. Local derivatives $\partial v/\partial a = b$, $\partial v/\partial b = a$.

$$\overline{a} = \overline{v} \cdot b, \qquad \overline{b} = \overline{v} \cdot a.$$

Multiplication node: swap the inputs

D DERIVATION

Node $v = ab$. Local derivatives $\partial v/\partial a = b$, $\partial v/\partial b = a$.

$$\overline{a} = \overline{v} \cdot b, \qquad \overline{b} = \overline{v} \cdot a.$$

$\boxed{\times \text{ sends the other input}}$

Multiplication node: swap the inputs

D DERIVATION

Node $v = ab$. Local derivatives $\partial v/\partial a = b$, $\partial v/\partial b = a$.

$$\overline{a} = \overline{v} \cdot b, \qquad \overline{b} = \overline{v} \cdot a.$$

$\times$ **sends the other input**

If $a = 2, b = 5, \overline{v} = 3$: then $\overline{a} = 3 \cdot 5 = 15$ and $\overline{b} = 3 \cdot 2 = 6$.

Activation node

D DERIVATION

Node $v = \varphi(u)$ (elementwise nonlinearity). Local derivative $\partial v/\partial u = \varphi'(u)$.

$\overline{u} = \overline{v} \cdot \varphi'(u)$.

Activation node

D DERIVATION

Node $v = \varphi(u)$ (elementwise nonlinearity). Local derivative $\partial v / \partial u = \varphi'(u)$.

$\overline{u} = \overline{v} \cdot \varphi'(u)$.

sigmoid σ: $\varphi'(u) = \sigma(u)(1 - \sigma(u))$

ReLU: $\varphi'(u) = \mathbb{1}[u > 0]$

The rule table — memorize this

VISUAL

Forward op	Local grad	Backward update
$v = a + b$	$1, 1$	$\overline{a}$ += $\overline{v}$, $\overline{b}$ += $\overline{v}$
$v = ab$	b, a	$\overline{a}$ += $\overline{v}b$, $\overline{b}$ += $\overline{v}a$
$v = u^2$	$2u$	$\overline{u}$ += $\overline{v} \cdot 2u$
$v = \varphi(u)$	$\varphi'(u)$	$\overline{u}$ += $\overline{v}\varphi'(u)$

The rule table — memorize this

V VISUAL

Forward op	Local grad	Backward update
$v = a + b$	$1, 1$	$\overline{a}$ += $\overline{v}$, $\overline{b}$ += $\overline{v}$
$v = ab$	b, a	$\overline{a}$ += $\overline{v}b$, $\overline{b}$ += $\overline{v}a$
$v = u^2$	$2u$	$\overline{u}$ += $\overline{v} \cdot 2u$
$v = \varphi(u)$	$\varphi'(u)$	$\overline{u}$ += $\overline{v}\varphi'(u)$

Use +=, not = — a variable feeding several nodes accumulates gradient from **all** of them.

Gradients accumulate at branches

One variable, many paths

D DERIVATION

If x influences $\mathcal{L}$ through several intermediate variables $u_1, \ldots, u_k$:

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{i=1}^{k} \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial x}.$$

One variable, many paths

D DERIVATION

If x influences $\mathcal{L}$ through several intermediate variables $u_1, \ldots, u_k$:

$$\frac{\partial \mathcal{L}}{\partial x} = \sum_{i=1}^{k} \frac{\partial \mathcal{L}}{\partial u_i} \frac{\partial u_i}{\partial x}.$$

gradients from every path add

Gradients add at branches

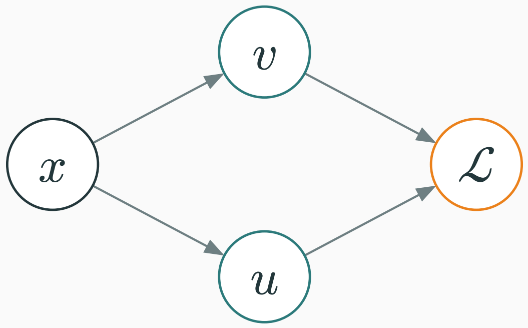
V VISUAL

When x fans out, its gradient is the **sum** over out-edges:

Gradients add at branches

V VISUAL

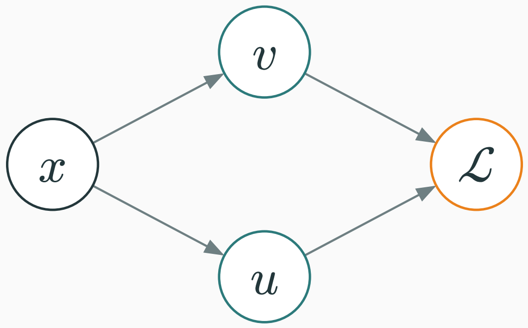
When x fans out, its gradient is the **sum** over out-edges:



Gradients add at branches

V VISUAL

When x fans out, its gradient is the **sum** over out-edges:



$$\overline{x} = \overline{x}_{\text{via } u} + \overline{x}_{\text{via } v}.$$

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x.$

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x$. By direct calculus:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}x} = 2x + 3, \quad \text{at } x = 4: \quad 2(4) + 3 = 11.$$

Worked branch example

Let $u = x^2, \;\; v = 3x, \;\; \mathcal{L} = u + v = x^2 + 3x$. By direct calculus:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}x} = 2x + 3, \quad \text{at } x = 4: \quad 2(4) + 3 = 11.$$

Now let us get the **same** answer by backprop.

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28.$

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28$. **Backward:** seed $\overline{\mathcal{L}} = 1$; add node gives $\overline{u} = 1, \overline{v} = 1$.

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28$. **Backward:** seed $\overline{\mathcal{L}} = 1$; add node gives $\overline{u} = 1, \overline{v} = 1$.

$$\overline{x}_{\text{via } u} = \overline{u} \cdot 2x = 1 \cdot 8 = 8, \quad \overline{x}_{\text{via } v} = \overline{v} \cdot 3 = 3.$$

Same answer, by backprop

D DERIVATION

Forward at $x = 4$: $u = 16,\ v = 12,\ \mathcal{L} = 28$. **Backward:** seed $\overline{\mathcal{L}} = 1$; add node gives $\overline{u} = 1, \overline{v} = 1$.

$$\overline{x}_{\text{via } u} = \overline{u} \cdot 2x = 1 \cdot 8 = 8, \quad \overline{x}_{\text{via } v} = \overline{v} \cdot 3 = 3.$$

$$\overline{x} = 8 + 3 = 11. \quad \checkmark$$

Why += matters

Both paths write into the **same** $\overline{x}$. If the second overwrote the first, we would lose the 8.

Why += matters

Both paths write into the **same** $\bar{x}$. If the second overwrote the first, we would lose the 8.

> In PyTorch, `.grad` **accumulates** across `backward()` calls — that is exactly this rule. You must `optimizer.zero_grad()` each step, or gradients from old steps pile up.

Checkpoint: gradient accumulation

Q QUESTION

Why is a branch node's backward update written with `+=` rather than `=`?

A. It makes the forward pass faster

B. The variable receives a contribution from every downstream path

C. It changes multiplication into addition

D. Autodiff cannot store scalar values

Answer: gradient accumulation

A ANSWER

Correct: B — The variable receives a contribution from every downstream path

Why: The chain rule sums all paths from the variable to the loss; assignment would discard an earlier contribution.

The central worked example

A scalar linear model

D DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$.

A scalar linear model

D DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$. Concrete numbers:

$$x = 3, \quad w = 2, \quad b = 1, \quad y = 10.$$

A scalar linear model

D DERIVATION

Predict $\hat{y} = wx + b$, squared-error loss $\mathcal{L} = (\hat{y} - y)^2$. Concrete numbers:

$$x = 3, \quad w = 2, \quad b = 1, \quad y = 10.$$

Four scalars, one loss. We will get $\partial\mathcal{L}$ w.r.t. **all four** by hand, then check with PyTorch.

Break into primitives

Decompose the loss into elementary nodes:

$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

Break into primitives

Decompose the loss into elementary nodes:

$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

$\boxed{\times\textbf{, then } +\textbf{, then } -\textbf{, then } (\cdot)^2}$

Break into primitives

Decompose the loss into elementary nodes:

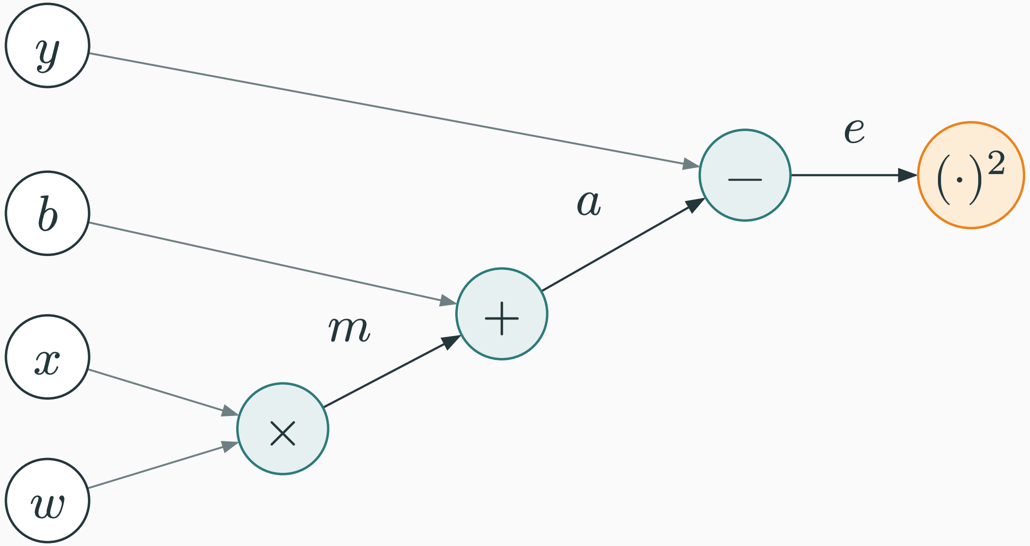
$$m = wx, \quad a = m + b, \quad e = a - y, \quad \mathcal{L} = e^2.$$

$\times$**, then** $+$**, then** $-$**, then** $(\cdot)^2$

Each is one of the nodes from our rule table.

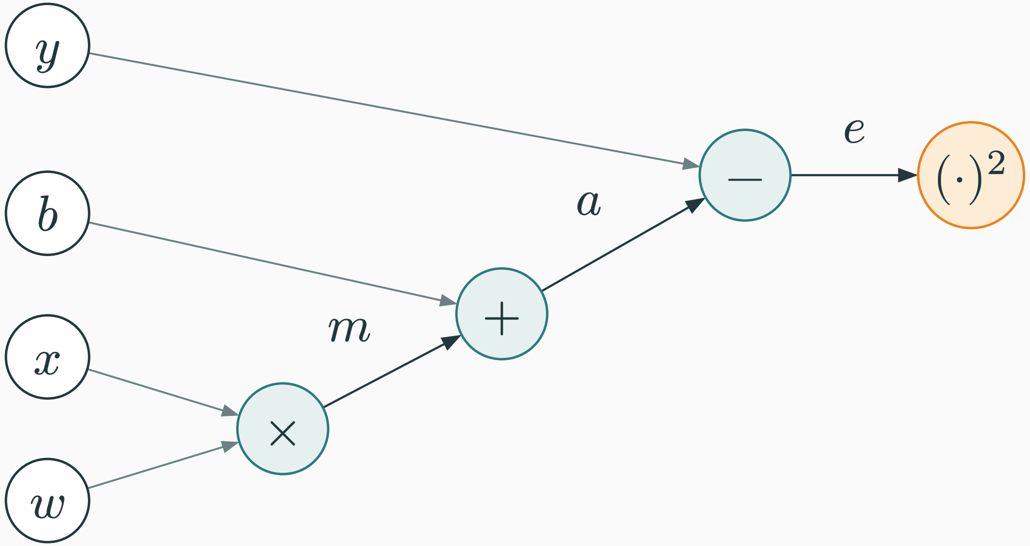
The computation graph

VISUAL



The computation graph

V VISUAL



inputs (ink) → ops $\times$, $+$, $-$ (teal) → loss $(\cdot)^2$ (orange)

Forward pass

D DERIVATION

Node	Rule	Value
$m = wx$	$2 \cdot 3$	6
$a = m + b$	$6 + 1$	7
$e = a - y$	$7 - 10$	-3
$\mathcal{L} = e^2$	$(-3)^2$	9

Forward pass

D DERIVATION

Node	Rule	Value
$m = wx$	$2 \cdot 3$	6
$a = m + b$	$6 + 1$	7
$e = a - y$	$7 - 10$	-3
$\mathcal{L} = e^2$	$(-3)^2$	9

$\mathcal{L} = 9$ **— now run the graph backward**

Backward: seed and the square node

D DERIVATION

Seed the output: $\overline{\mathcal{L}} = \partial\mathcal{L}/\partial\mathcal{L} = 1$.

Backward: seed and the square node

D DERIVATION

Seed the output: $\overline{\mathcal{L}} = \partial\mathcal{L}/\partial\mathcal{L} = 1$. Square node $\mathcal{L} = e^2$, local $\partial\mathcal{L}/\partial e = 2e$:

$$\overline{e} = \overline{\mathcal{L}} \cdot 2e = 1 \cdot 2(-3) = -6.$$

Backward: the subtraction node

D DERIVATION

Node $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

Backward: the subtraction node

D DERIVATION

Node $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

$$\overline{a} = \overline{e} \cdot 1 = -6$$

Backward: the subtraction node

D DERIVATION

Node $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

$$\bar{a} = \bar{e} \cdot 1 = -6$$

$$\bar{y} = \bar{e} \cdot (-1) = 6$$

Backward: the subtraction node

D DERIVATION

Node $e = a - y$, locals $\partial e/\partial a = 1$, $\partial e/\partial y = -1$:

$$\bar{a} = \bar{e} \cdot 1 = -6$$

$$\bar{y} = \bar{e} \cdot (-1) = 6$$

The target y already has a gradient — we would use $\bar{y}$ if y were itself a learned quantity.

Backward: the addition node

D DERIVATION

Node $a = m + b$ **copies** its gradient:

$$\overline{m} = \overline{a} = -6, \qquad \overline{b} = \overline{a} = -6.$$

Backward: the addition node

D DERIVATION

Node $a = m + b$ **copies** its gradient:

$$\overline{m} = \overline{a} = -6, \qquad \overline{b} = \overline{a} = -6.$$

So $\overline{b} = -6$ is one of our final answers.

Backward: the multiplication node

D DERIVATION

Node $m = wx$ **sends the other input**:

Backward: the multiplication node

D DERIVATION

Node $m = wx$ **sends the other input**:

$$\overline{w} = \overline{m} \cdot x = -6 \cdot 3 = -18$$

Backward: the multiplication node

D DERIVATION

Node $m = wx$ **sends the other input**:

$$\overline{w} = \overline{m} \cdot x = -6 \cdot 3 = -18$$

$$\overline{x} = \overline{m} \cdot w = -6 \cdot 2 = -12$$

Backward: the multiplication node

D DERIVATION

Node $m = wx$ **sends the other input**:

$$\overline{w} = \overline{m} \cdot x = -6 \cdot 3 = -18$$

$$\overline{x} = \overline{m} \cdot w = -6 \cdot 2 = -12$$

$$\boxed{\overline{w} = -18, \quad \overline{x} = -12}$$

Final gradients

V VISUAL

$\partial\mathcal{L}/\partial w$	$\partial\mathcal{L}/\partial b$	$\partial\mathcal{L}/\partial x$	$\partial\mathcal{L}/\partial y$
-18	-6	-12	6

Final gradients

V VISUAL

$\partial\mathcal{L}/\partial w$	$\partial\mathcal{L}/\partial b$	$\partial\mathcal{L}/\partial x$	$\partial\mathcal{L}/\partial y$
-18	-6	-12	6

One backward sweep produced **all four** partial derivatives — no formula re-derivation per parameter.

The backward pass, visualized

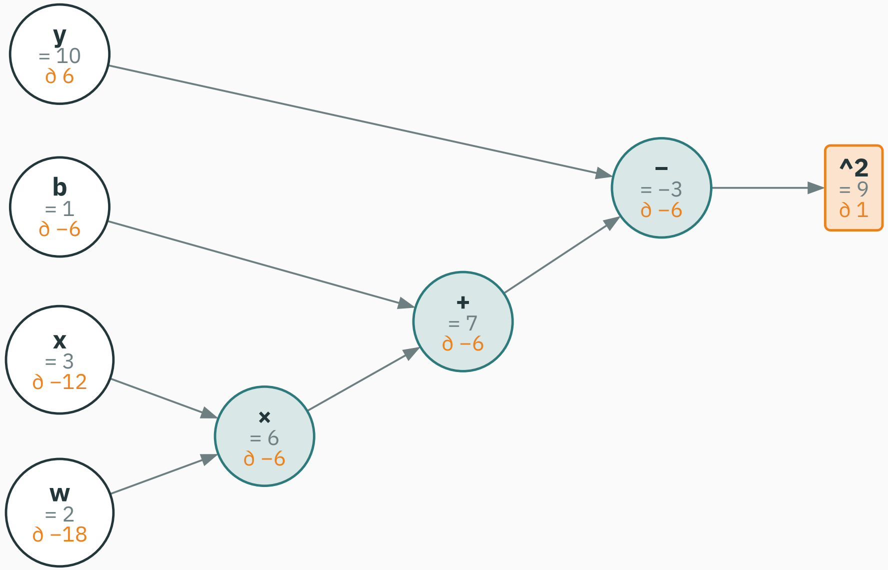
V VISUAL

Each node shows its forward value and its backward adjoint, **drawn by our autodiff from the real computation** — nothing typed by hand:

The backward pass, visualized

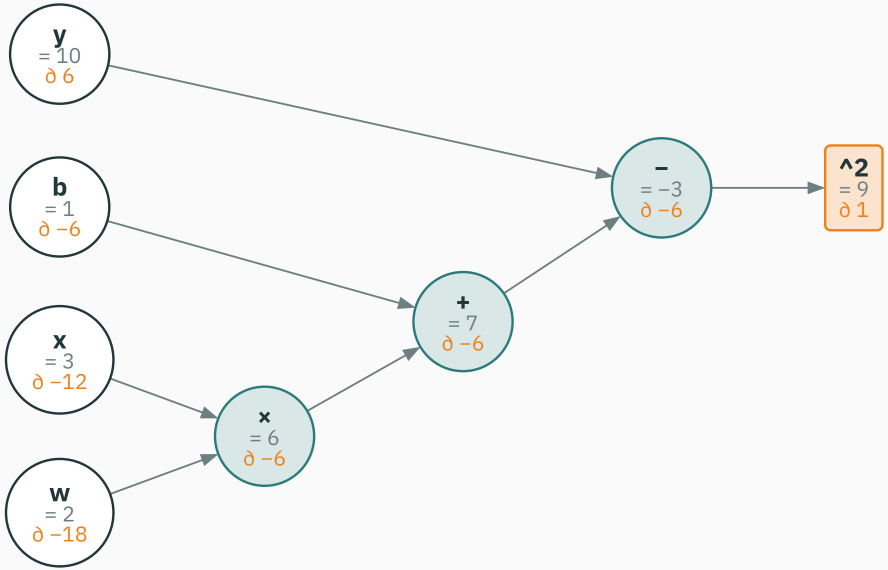
V VISUAL

Each node shows its forward value and its backward adjoint, **drawn by our autodiff from the real computation** — nothing typed by hand:



The backward pass, visualized

Each node shows its forward value and its backward adjoint, **drawn by our autodiff from the real computation** — nothing typed by hand:



$\overline{w} = -18, \overline{x} = -12, \overline{b} = -6, \overline{y} = 6$ — one reverse sweep, right to left

Check with direct calculus

D DERIVATION

Differentiate $\mathcal{L} = (wx + b - y)^2$ directly. With $wx + b - y = -3$:

Check with direct calculus

D DERIVATION

Differentiate $\mathcal{L} = (wx + b - y)^2$ directly. With $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

Check with direct calculus

D DERIVATION

Differentiate $\mathcal{L} = (wx + b - y)^2$ directly. With $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

Check with direct calculus

D DERIVATION

Differentiate $\mathcal{L} = (wx + b - y)^2$ directly. With $wx + b - y = -3$:

$$\frac{\partial \mathcal{L}}{\partial w} = 2(wx + b - y)x = 2(-3)(3) = -18 \quad \checkmark$$

$$\frac{\partial \mathcal{L}}{\partial b} = 2(wx + b - y) = 2(-3) = -6 \quad \checkmark$$

backprop $=$ calculus — but mechanical & reusable

One gradient-descent step

D DERIVATION

Take $\eta = 0.1$ and update:

$$w \leftarrow 2 - 0.1(-18) = 3.8, \qquad b \leftarrow 1 - 0.1(-6) = 1.6.$$

One gradient-descent step

D DERIVATION

Take $\eta = 0.1$ and update:

$$w \leftarrow 2 - 0.1(-18) = 3.8, \qquad b \leftarrow 1 - 0.1(-6) = 1.6.$$

New prediction: $\hat{y} = 3.8 \cdot 3 + 1.6 = 13.$

One gradient-descent step

D DERIVATION

Take $\eta = 0.1$ and update:

$$w \leftarrow 2 - 0.1(-18) = 3.8, \qquad b \leftarrow 1 - 0.1(-6) = 1.6.$$

New prediction: $\hat{y} = 3.8 \cdot 3 + 1.6 = 13.$

Target is $y = 10$; we jumped from 7 to 13 — we **overshot** (both are off by 3).
Lecture 4 is about choosing η and the update rule so this is stable.

Interactive: the computation graph

INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Build a scalar computation graph, run the forward pass, then click **backward** to watch each adjoint $\overline{u}$ fill in — exactly the $(wx + b - y)^2$ example above.

Interactive: the computation graph

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Build a scalar computation graph, run the forward pass, then click **backward** to watch each adjoint $\overline{u}$ fill in — exactly the $(wx + b - y)^2$ example above.

Trace which local rule changes when the final squared-error operation is replaced with $|e|$.

From scalar graph to a neuron

A single neuron

Vector input, one output:

$$z = w^\top x + b, \quad a = \varphi(z), \quad \mathcal{L} = \ell(a, y).$$

A single neuron

Vector input, one output:

$$z = w^\top x + b, \quad a = \varphi(z), \quad \mathcal{L} = \ell(a, y).$$

same three nodes: affine → activation → loss

Backprop through the activation

D DERIVATION

Let $\delta := \overline{z} = \partial\mathcal{L}/\partial z$. Through $a = \varphi(z)$:

$$\delta = \overline{a} \cdot \varphi'(z), \quad \text{where } \overline{a} = \frac{\partial\mathcal{L}}{\partial a}.$$

Backprop through the activation

D DERIVATION

Let $\delta := \overline{z} = \partial\mathcal{L}/\partial z$. Through $a = \varphi(z)$:

$$\delta = \overline{a} \cdot \varphi'(z), \quad \text{where } \overline{a} = \frac{\partial\mathcal{L}}{\partial a}.$$

δ is the **error signal at the pre-activation**. Everything below the neuron is expressed through this one scalar.

Backprop through the affine part

D DERIVATION

Through $z = w^\top x + b$ with $\overline{z} = \delta$:

Backprop through the affine part

D DERIVATION

Through $z = w^\top x + b$ with $\overline{z} = \delta$:

$$\nabla_w \mathcal{L} = \delta\, x, \qquad \frac{\partial \mathcal{L}}{\partial b} = \delta, \qquad \overline{x} = \delta\, w.$$

Backprop through the affine part

D DERIVATION

Through $z = w^\top x + b$ with $\overline{z} = \delta$:

$$\nabla_w \mathcal{L} = \delta\, x, \qquad \frac{\partial \mathcal{L}}{\partial b} = \delta, \qquad \overline{x} = \delta\, w.$$

Parameter gradient $=$ error δ times the **input** x. This "δ times input" shape reappears at every layer.

Sigmoid + BCE: the clean cancellation

D DERIVATION

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$.

Sigmoid + BCE: the clean cancellation

D DERIVATION

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$.

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

Sigmoid + BCE: the clean cancellation

D DERIVATION

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$.

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\frac{\partial a}{\partial z} = a(1-a)$$

Sigmoid + BCE: the clean cancellation

D DERIVATION

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$.

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\frac{\partial a}{\partial z} = a(1-a)$$

Multiply — the denominators cancel:

$$\bar{z} = \frac{\partial \mathcal{L}}{\partial a} \cdot a(1-a) = a - y.$$

Sigmoid + BCE: the clean cancellation

D DERIVATION

Sigmoid $a = \sigma(z)$, binary cross-entropy $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$.

$$\frac{\partial \mathcal{L}}{\partial a} = -\frac{y}{a} + \frac{1-y}{1-a}$$

$$\frac{\partial a}{\partial z} = a(1-a)$$

Multiply — the denominators cancel:

$$\bar{z} = \frac{\partial \mathcal{L}}{\partial a} \cdot a(1-a) = a - y.$$

$\bar{z} = a - y$ **— prediction minus target**

Softmax + cross-entropy: same pattern

D DERIVATION

For multi-class, $p = \mathrm{softmax}(z)$ and $\mathcal{L} = -\log p_y$:

$\nabla_z \mathcal{L} = p - y,$

where y is the one-hot target.

Softmax + cross-entropy: same pattern

D DERIVATION

For multi-class, $p = \mathrm{softmax}(z)$ and $\mathcal{L} = -\log p_y$:

$$\nabla_z \mathcal{L} = p - y,$$

where y is the one-hot target.

$\boxed{\bar{z} = p - y \textbf{ — the identical “prediction} - \textbf{target” form}}$

Softmax + cross-entropy: same pattern

D DERIVATION

For multi-class, $p = \mathrm{softmax}(z)$ and $\mathcal{L} = -\log p_y$:

$$\nabla_z \mathcal{L} = p - y,$$

where y is the one-hot target.

$\boxed{\bar{z} = p - y \text{ — the identical “prediction} - \text{target” form}}$

Choosing the loss to **match** the output nonlinearity makes the output-layer gradient trivially simple.

The dense layer

Forward: shapes matter

D DERIVATION

A dense layer maps $x \in \mathbb{R}^n$ to $z \in \mathbb{R}^m$:

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m.$$

Forward: shapes matter

D DERIVATION

A dense layer maps $x \in \mathbb{R}^n$ to $z \in \mathbb{R}^m$:

$$z = Wx + b, \quad W \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m.$$

Read the shapes off the equation: W has one row per output, one column per input.

Backward through a dense layer

D DERIVATION

Given upstream $\bar{z} \in \mathbb{R}^m$:

Backward through a dense layer

D DERIVATION

Given upstream $\overline{z} \in \mathbb{R}^m$:

$$\nabla_W \mathcal{L} = \overline{z}\, x^\top \quad (m \times n), \qquad \nabla_b \mathcal{L} = \overline{z}, \qquad \overline{x} = W^\top \overline{z} \quad (n).$$

Backward through a dense layer

D DERIVATION

Given upstream $\overline{z} \in \mathbb{R}^m$:

$$\nabla_W \mathcal{L} = \overline{z}\, x^\top \quad (m \times n), \qquad \nabla_b \mathcal{L} = \overline{z}, \qquad \overline{x} = W^\top \overline{z} \quad (n).$$

$$\boxed{\nabla_W = \overline{z} x^\top \bullet \ \overline{x} = W^\top \overline{z}}$$

Batch version

DERIVATION

Stack N examples as rows: $X \in \mathbb{R}^{N\times n}$, $Z = XW^\top + \mathbb{1}b^\top$.

Batch version

DERIVATION

Stack N examples as rows: $X \in \mathbb{R}^{N\times n}$, $Z = XW^\top + \mathbb{1}b^\top$. Given $\overline{Z} \in \mathbb{R}^{N\times m}$:

$$\nabla_W \mathcal{L} = \overline{Z}^\top X, \qquad \nabla_b \mathcal{L} = \sum_{i=1}^{N} \overline{Z}_i.$$

Batch version

D DERIVATION

Stack N examples as rows: $X \in \mathbb{R}^{N\times n}$, $Z = XW^\top + \mathbb{1}b^\top$. Given $\overline{Z} \in \mathbb{R}^{N\times m}$:

$$\nabla_W \mathcal{L} = \overline{Z}^\top X, \qquad \nabla_b \mathcal{L} = \sum_{i=1}^{N} \overline{Z}_i.$$

Summing rows of $\overline{Z}$ for ∇_b is just the branch rule: b is **shared** across all N examples, so its gradients accumulate.

Shape-checking is free debugging

V VISUAL

Quantity	Shape	Must match
W	$m \times n$	∇_W
b	m	∇_b
x	n	$\overline{x}$
z	m	$\overline{z}$

Shape-checking is free debugging

V VISUAL

Quantity	Shape	Must match
W	$m \times n$	∇_W
b	m	∇_b
x	n	$\overline{x}$
z	m	$\overline{z}$

gradient shape $=$ parameter shape — always

Backprop through a two-layer MLP

Forward pass

D DERIVATION

$$z^{(1)} = W_1 x + b_1, \quad h = \varphi\big(z^{(1)}\big),$$

$$z^{(2)} = W_2 h + b_2, \quad p = \mathrm{softmax}\big(z^{(2)}\big), \quad \mathcal{L} = -\log p_y.$$

Forward pass

$$z^{(1)} = W_1 x + b_1, \quad h = \varphi\big(z^{(1)}\big),$$

$$z^{(2)} = W_2 h + b_2, \quad p = \mathrm{softmax}\big(z^{(2)}\big), \quad \mathcal{L} = -\log p_y.$$

affine → activation → affine → softmax → CE

The forward graph

V VISUAL

$$x \rightarrow z^{(1)} \rightarrow h \rightarrow z^{(2)} \rightarrow p \rightarrow \mathcal{L}$$

The forward graph

V VISUAL

$$x \to z^{(1)} \to h \to z^{(2)} \to p \to \mathcal{L}$$

backprop reverses every arrow, right to left

Backward: the output layer

D DERIVATION

Softmax + CE gives the clean output-layer signal:

$$\overline{z}^{(2)} = p - y.$$

Backward: the output layer

D DERIVATION

Softmax + CE gives the clean output-layer signal:

$$\overline{z}^{(2)} = p - y.$$

Then, using the dense-layer rules:

$$\nabla_{W_2}\mathcal{L} = \overline{z}^{(2)} h^\top, \qquad \overline{h} = W_2^\top \overline{z}^{(2)}.$$

Backward: the hidden and first layers

D DERIVATION

Through the activation $h = \varphi\big(z^{(1)}\big)$ (elementwise):

$$\overline{z}^{(1)} = \overline{h} \odot \varphi'\big(z^{(1)}\big).$$

Backward: the hidden and first layers

D DERIVATION

Through the activation $h = \varphi(z^{(1)})$ (elementwise):

$$\overline{z}^{(1)} = \overline{h} \odot \varphi'\left(z^{(1)}\right).$$

Then the first affine layer:

$$\nabla_{W_1}\mathcal{L} = \overline{z}^{(1)} x^\top, \qquad \nabla_{b_1}\mathcal{L} = \overline{z}^{(1)}.$$

Backward: the hidden and first layers

D DERIVATION

Through the activation $h = \varphi\big(z^{(1)}\big)$ (elementwise):

$$\overline{z}^{(1)} = \overline{h} \odot \varphi'\big(z^{(1)}\big).$$

Then the first affine layer:

$$\nabla_{W_1}\mathcal{L} = \overline{z}^{(1)} x^\top, \qquad \nabla_{b_1}\mathcal{L} = \overline{z}^{(1)}.$$

$\odot$ is elementwise (Hadamard) — the activation Jacobian is diagonal.

The complete recipe

V VISUAL

Forward	**Backward**
$z^{(1)} = W_1 x + b_1$	$\overline{z}^{(1)} = \overline{h} \odot \varphi'\big(z^{(1)}\big)$
$h = \varphi\big(z^{(1)}\big)$	$\nabla_{W_1} = \overline{z}^{(1)} x^\top, \ \nabla_{b_1} = \overline{z}^{(1)}$
$z^{(2)} = W_2 h + b_2$	$\overline{h} = W_2^\top \overline{z}^{(2)}$
$p = \mathrm{softmax}\big(z^{(2)}\big)$	$\nabla_{W_2} = \overline{z}^{(2)} h^\top, \ \nabla_{b_2} = \overline{z}^{(2)}$
$\mathcal{L} = -\log p_y$	$\overline{z}^{(2)} = p - y$

The complete recipe

V VISUAL

Forward	**Backward**
$z^{(1)} = W_1 x + b_1$	$\overline{z}^{(1)} = \overline{h} \odot \varphi'\big(z^{(1)}\big)$
$h = \varphi\big(z^{(1)}\big)$	$\nabla_{W_1} = \overline{z}^{(1)} x^\top, \ \nabla_{b_1} = \overline{z}^{(1)}$
$z^{(2)} = W_2 h + b_2$	$\overline{h} = W_2^\top \overline{z}^{(2)}$
$p = \mathrm{softmax}\big(z^{(2)}\big)$	$\nabla_{W_2} = \overline{z}^{(2)} h^\top, \ \nabla_{b_2} = \overline{z}^{(2)}$
$\mathcal{L} = -\log p_y$	$\overline{z}^{(2)} = p - y$

same two moves at every layer: $\delta x^\top$ for weights, $W^\top \delta$ downstream

Interactive: MLP backprop

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Step through a small MLP's forward pass, then watch $\overline{z}^{(2)} = p - y$ propagate back through W_2, the activation, and W_1 — the adjoints light up layer by layer.

Interactive: MLP backprop

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/autograd

Step through a small MLP's forward pass, then watch $\overline{z}^{(2)} = p - y$ propagate back through W_2, the activation, and W_1 — the adjoints light up layer by layer.

Trace the same upstream adjoint into the parameter and input branches of the affine layer.

Checkpoint: dense-layer backward pass

Q QUESTION

For $z = Wx + b$, which pair of gradients uses the same upstream adjoint $\bar{z}$?

A. $\nabla_W = \bar{z}x^\top$ and $\bar{x} = W^\top \bar{z}$

B. $\nabla_W = W^\top x$ and $\bar{x} = \bar{z}x^\top$

C. $\nabla_W = x$ and $\bar{x} = W$

D. $\nabla_W = 0$ and $\bar{x} = 0$

Answer: dense-layer backward pass

A ANSWER

Correct: A — $\nabla_W = \bar{z}x^\top$ and $\bar{x} = W^\top \bar{z}$

Why: Both branches apply the chain rule to the same output adjoint; they differ only in the local derivative of the affine operation.

Automatic differentiation

Backprop is reverse-mode autodiff

Backprop is not two things you might guess:

Backprop is reverse-mode autodiff

Backprop is not two things you might guess:

not symbolic differentiation — no giant closed-form expression

not finite differences — no ε-perturbations

Backprop is reverse-mode autodiff

Backprop is not two things you might guess:

not symbolic differentiation — no giant closed-form expression

not finite differences — no ε-perturbations

it is reverse-mode automatic differentiation: the chain rule applied to the executed program

Why reverse mode?

D DERIVATION

A loss is a map $f : \mathbb{R}^P \to \mathbb{R}$: **many** inputs, **one** output.

Why reverse mode?

D DERIVATION

A loss is a map $f : \mathbb{R}^P \to \mathbb{R}$: **many** inputs, **one** output.

Forward mode: one pass **per input** → $O(P)$.

Reverse mode: one pass **per output** → $O(1)$.

Why reverse mode?

D DERIVATION

A loss is a map $f : \mathbb{R}^P \to \mathbb{R}$: **many** inputs, **one** output.

Forward mode: one pass **per input** → $O(P)$.

Reverse mode: one pass **per output** → $O(1)$.

$P \approx 10^9$ **inputs, one scalar loss** $\Rightarrow$ **reverse mode wins**

What PyTorch / JAX actually do

During the forward pass the framework:

What PyTorch / JAX actually do

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);

What PyTorch / JAX actually do

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);
- **stores** intermediate values (needed by local derivatives);

What PyTorch / JAX actually do

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);
- **stores** intermediate values (needed by local derivatives);
- on `backward()`: seeds $\overline{\mathcal{L}} = 1$, applies each op's local rule, and **accumulates** into `.grad`.

What PyTorch / JAX actually do

During the forward pass the framework:

- **records** each primitive op and its inputs onto a tape (the graph);
- **stores** intermediate values (needed by local derivatives);
- on `backward()`: seeds $\overline{\mathcal{L}} = 1$, applies each op's local rule, and **accumulates** into `.grad`.

$$\boxed{\textbf{you write only the forward pass — the backward pass is generated}}$$

PyTorch: the scalar example

INTERACTIVE

```
import torch
x = torch.tensor(3.0)
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(10.0)

L = (w*x + b - y)**2      # forward:  L = 9
L.backward()              # reverse-mode autodiff

print(L.item())           # 9.0
print(w.grad, b.grad)     # tensor(-18.)  tensor(-6.)
```

PyTorch: the scalar example

I INTERACTIVE

```
import torch
x = torch.tensor(3.0)
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(10.0)

L = (w*x + b - y)**2      # forward:  L = 9
L.backward()              # reverse-mode autodiff

print(L.item())           # 9.0
print(w.grad, b.grad)     # tensor(-18.)  tensor(-6.)
```

matches our hand computation exactly

The same graph, in **this** deck

I INTERACTIVE

Our own toolkit ships a tiny reverse-mode autodiff (chalkdust `autodiff`) — build the expression, get **every** gradient in one backward pass. This slide runs it:

The same graph, in **this** deck

I INTERACTIVE

Our own toolkit ships a tiny reverse-mode autodiff (chalkdust `autodiff`) — build the expression, get **every** gradient in one backward pass. This slide runs it:

```
#import "@local/autodiff:0.1.0" as ad
#let L = ad.expr("(w*x + b - y)^2", ("w", "x", "b", "y"))
#ad.value(L, (2, 3, 1, 10))   // the loss
#ad.grad(L,  (2, 3, 1, 10))   // (w̄, x̄, b̄, ȳ), one backward pass
```

The same graph, in **this** deck

I INTERACTIVE

Our own toolkit ships a tiny reverse-mode autodiff (chalkdust `autodiff`) — build the expression, get **every** gradient in one backward pass. This slide runs it:

```
#import "@local/autodiff:0.1.0" as ad
#let L = ad.expr("(w*x + b - y)^2", ("w", "x", "b", "y"))
#ad.value(L, (2, 3, 1, 10))   // the loss
#ad.grad(L,  (2, 3, 1, 10))   // (w̄, x̄, b̄, ȳ), one backward pass
```

computed live in this slide: $\mathcal{L} = 9$, $\overline{w} = -18, \overline{x} = -12, \overline{b} = -6, \overline{y} = 6$ **— the same four numbers, by hand, by PyTorch, and by our own graph.**

Gradients accumulate in frameworks

`.grad` is **added to**, never reset, by `backward()`. The standard loop is:

Gradients accumulate in frameworks

`.grad` is **added to**, never reset, by `backward()`. The standard loop is:

```
for x, y in loader:
    optimizer.zero_grad()      # clear old gradients
    loss = criterion(model(x), y)
    loss.backward()            # accumulate this batch's gradients
    optimizer.step()           # theta <- theta - eta * grad
```

Gradients accumulate in frameworks

`.grad` is **added to**, never reset, by `backward()`. The standard loop is:

```
for x, y in loader:
    optimizer.zero_grad()       # clear old gradients
    loss = criterion(model(x), y)
    loss.backward()             # accumulate this batch's gradients
    optimizer.step()            # theta <- theta - eta * grad
```

Forget `zero_grad()` and you sum gradients across steps — the same `+=` branch rule, now biting you.

Gradient checking & gradient flow

Finite-difference gradient check

D DERIVATION

To **debug** an analytic gradient, compare against a central difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\theta + \varepsilon e_j) - \mathcal{L}(\theta - \varepsilon e_j)}{2\varepsilon}.$$

Finite-difference gradient check

D DERIVATION

To **debug** an analytic gradient, compare against a central difference:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} \approx \frac{\mathcal{L}(\theta + \varepsilon e_j) - \mathcal{L}(\theta - \varepsilon e_j)}{2\varepsilon}.$$

Great for **unit tests** ($\varepsilon \approx 10^{-5}$), far too slow for training — one loss evaluation **per parameter**.

Gradient flow in deep chains

D DERIVATION

Stack L layers $h_0 \to h_1 \to \dots \to h_L$. The chain rule multiplies Jacobians:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^{L} \frac{\partial h_\ell}{\partial h_{\ell-1}}.$$

Gradient flow in deep chains

D DERIVATION

Stack L layers $h_0 \to h_1 \to \ldots \to h_L$. The chain rule multiplies Jacobians:

$$\frac{\partial \mathcal{L}}{\partial h_0} = \frac{\partial \mathcal{L}}{\partial h_L} \prod_{\ell=1}^{L} \frac{\partial h_\ell}{\partial h_{\ell-1}}.$$

a product of L terms — it can vanish or explode

Sigmoid saturation

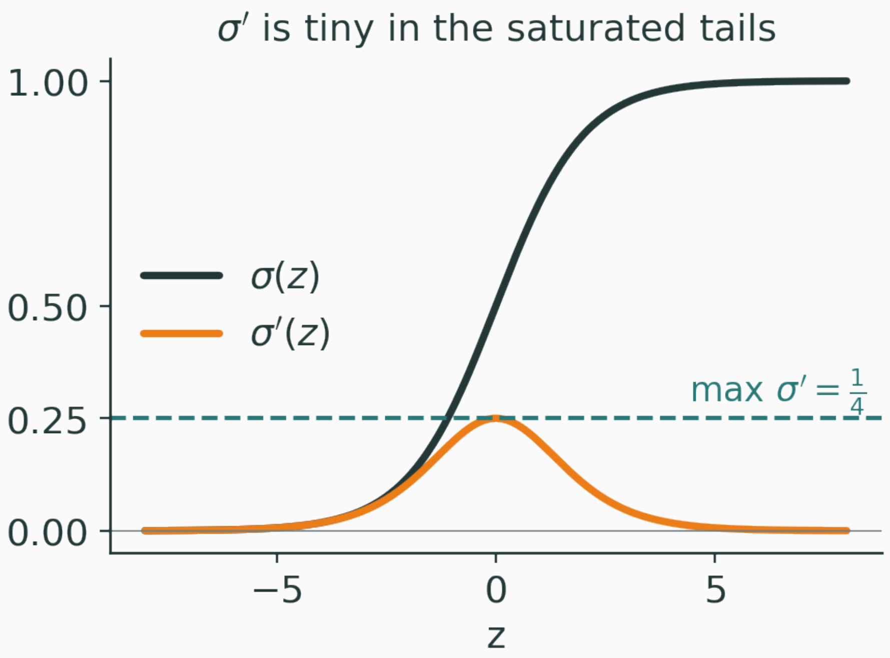
V VISUAL

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:

Sigmoid saturation

V VISUAL

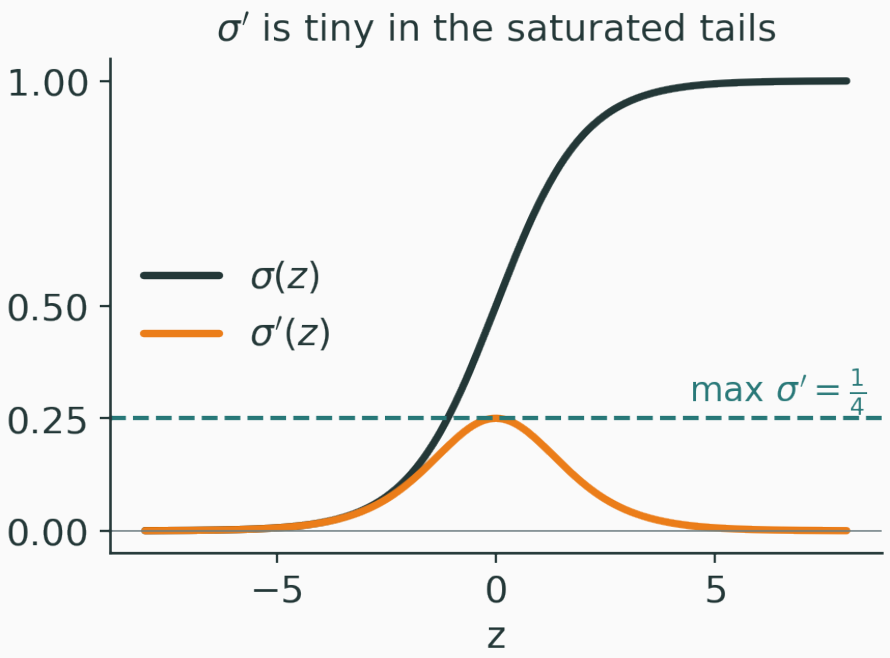
$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$, and ≈ 0 for large $|z|$:



Sigmoid saturation

V VISUAL

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \le \frac{1}{4}$, and ≈ 0 for large $|z|$:



each sigmoid layer multiplies the gradient by at most $1/4$

Vanishing gradients

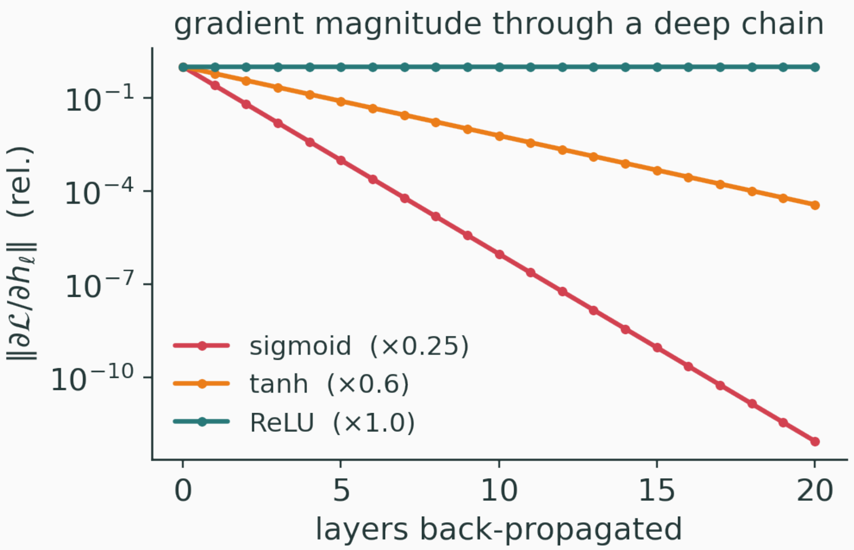
V VISUAL

Multiply many small local derivatives → the signal **decays** with depth:

Vanishing gradients

V VISUAL

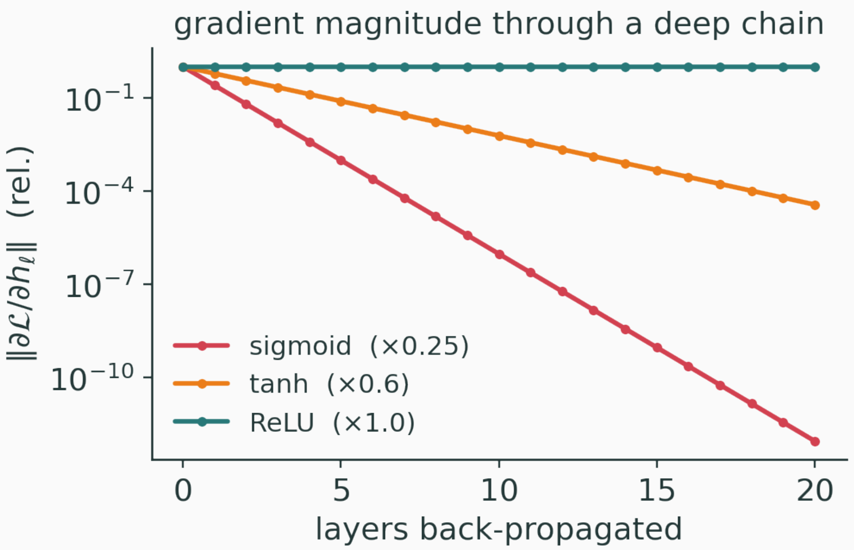
Multiply many small local derivatives → the signal **decays** with depth:



Vanishing gradients

V VISUAL

Multiply many small local derivatives → the signal **decays** with depth:



Sigmoid decays fast; ReLU (local slope 0 or 1) keeps gradients alive.

Interactive: gradient flow

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Interactive: gradient flow

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/vanishing-gradients

Sweep depth and activation function; watch the gradient magnitude at layer 0 collapse for sigmoid and survive for ReLU.

Sweep depth to see repeated small local slopes rapidly erase an early-layer gradient.

Summary

Backprop in one sentence

Seed $\overline{\mathcal{L}} = 1$, then walk the graph **backward**, at each node computing

$$\textbf{downstream} = \textbf{upstream} \times \textbf{local},$$

and **adding** gradients wherever a variable branched.

The formulas to keep

V VISUAL

Setting	**Backward rule**
scalar node	$\overline{u} = \overline{v} \cdot \partial v / \partial u$
dense layer	$\nabla_W = \overline{z} x^\top, \quad \overline{x} = W^\top \overline{z}$
activation	$\overline{z} = \overline{a} \odot \varphi'(z)$
softmax + CE	$\overline{z} = p - y$
branch point	gradients **add**

What comes next

We can now compute $\nabla_\theta \mathcal{L}$ for **any** network, exactly and cheaply.

What comes next

We can now compute $\nabla_\theta \mathcal{L}$ for **any** network, exactly and cheaply.

Next (Lecture 4): how to use it — SGD, momentum, Adam

What comes next

We can now compute $\nabla_\theta \mathcal{L}$ for **any** network, exactly and cheaply.

Next (Lecture 4): how to use it — SGD, momentum, Adam

Backprop gives the **direction**; optimization decides the **step**.

We can compute $\nabla_\theta \mathcal{L}$.

Next: how to **use** it — SGD, momentum, Adam.